

Lab 5

Semantic Errors Part B

NOTE:

Be sure to read the “Submitting your lab” section at the end of this document **before** starting work on this lab.

Purpose:

In this lab, you will add more semantic error checking to your compiler using the Visitor pattern.

NOTE:

For this lab, you are not allowed to change any of the AST node classes except in the following two instances:

1. When specifically instructed to do so in these directions.
2. To add getters. The getters are only allowed to get a value, they are not allowed to do any calculations, and they are not allowed to update any objects. Sometimes “get a value” requires chaining getters. That is acceptable, but no calculations are allowed.

Preliminaries:

We will be using the Visitor pattern to accomplish this week’s task. I’ve made a minor change to `cVisitor.h`, which will make your life easier. The `VisitAllNodes` method is no longer pure virtual. You need to update the declaration in `cVisitor.h` and add the following implementation to `cVisitor.cpp`:

```
void cVisitor::VisitAllNodes(cAstNode *node)
{ node->Visit(this); }
```

I’ve also provided a new `main.cpp`. Use the new version to intelligently update yours to create and invoke the semantic visitor. Note that the new `main()` I supplied will work for labs 5a, 6, and 7. There are `#define` values that indicate what lab features should be included in a particular build. For this week `LAB5B` should be defined.

The type of expressions

If you haven’t already, add a pure virtual `GetType()` to the `cExprNode` base class:

```
cDeclNode *GetType() = 0;
```

You will have to override the method in each class that inherits from `cExprNode`.

The name of things:

When printing error messages, it is handy to have a convenient way of getting the name of something. I'd suggest adding a pure virtual function to `cDeclNode`:

```
string GetName() = 0;
```

Basic visitor

You will need to create a class that subclasses `cVisitor`. This is the class where ALL the processing for this week's lab will take place.

A few hints: these are only hints; you do not need to follow my method.

1. In my `cVisitor.h` and `cVisitor.cpp`, I keep all the methods in alphabetic order by AST node name. This makes it easy to check if I have them all. If yours doesn't yet have all your nodes, be sure to add them at this time.
2. In my subclass, I include all the methods even though I don't need to implement them all. I comment them all out when I first create the class and then uncomment them as I need them. This helps me remember what I've done, and what I might need to do in the future. Feel free to only create the methods you need instead of having them all there but commented out.

The Tests

The test numbers roughly follow the following sections. The tests are intended to run out of a directory called `test5b`.

1) Type compatibility

In order to facilitate comparing types, it would be handy to have a single function that performs this check. The logical place for such a function is in the `cDeclNode` class:

```
bool IsCompatibleWith(cDeclNode *type);
```

The function should return true if the `type` is compatible with `this`.

When you find a mismatched assignment, you should emit the following error:

```
Cannot assign <type> to <type>
```

The first place to add code for checking this is in the visitor for `cAssignNode`. Remember: the code for checking semantic errors in this lab must ALL be in the subclass of `cVisitor`.

You also need to determine the type of expressions. For Binary and Unary expressions, the type is based on the type of the operands. For Unary, it is simply the type of the only operand. For Binary, you need to determine the "largest" type. This is not based solely on size. In an expression with different types, within a category (integer, floating), the smaller promotes to the larger. Between categories, the integer promotes to the floating-point type.

The one exception to the promotion rules is that relational operators (> < >=<= == !=) and logical operators (|| &&) always return an `int`.

2) Illegal Operations

You cannot use a function name as if it was a variable. For example, the following code is not legal:

```
program
{
    int foo() { return 1; }
    int a;
    a = foo(); // legal: this is a function call
    a = foo;   // illegal, foo is not a variable
}
```

You must check for this everywhere that variables can be used, but this is easy to do by checking for this in the `cVarExprNode` visitor.

If you detect this error, you must emit the error:

```
Symbol <name> is a function, not a variable
```

The parent of every array reference must be an array. So, for example, in the `varref var[10]`, `var` must be an array. If it isn't you must emit the error:

```
<name> is not an array
```

Array indexes must be integers. If an array index is not an integer, you must emit the following error:

```
Index of <name> is not an int
```

Note: Some years I have the class implement arrays, other years I have the class implement structs. This year we will not do structs, so your code can make the simplifying assumption that all children of a `varref` (except the first one) are expressions. Rather than removing struct references from the grammar, we will treat struct references as undetected semantic errors.

3) Function calls

Functions must be called with the correct number of arguments. If not, the following error must be emitted:

```
<name> called with wrong number of arguments
```

The arguments in a function call must be assignment compatible with the formal arguments in the function declaration. If not, the following error must be emitted:

```
function <name> called with incompatible argument
```

Any function that gets called, must be fully defined. That is, there must be a function body in one of the declarations. If not, the following error must be emitted:

```
Function <name> not fully defined
```

Submitting your lab

Be sure to read these instructions. Your grade could be negatively affected if you do not follow them.

Part of the requirements for this lab are that you need to do all the processing in your Visitor subclass. I intend to use git to verify this. To facilitate this, you need to do the following:

1. Fix any Lab 5a issues before starting on this lab.
2. Commit and tag your “fixed” Lab 5a code
3. Provide the following tags as part of your submission:
 - a. The Lab 5a version I graded
 - b. Your “fixed” Lab 5a version
 - c. Your Lab 5b version

If you did not fix any Lab 5a issues, you can use the same tag for both the graded 5a and fixed 5a versions.

If you do not give me these tags, I will have to assume that you violated the rules and **take 15 points off your lab.**